

Reproducible Comparison of Specialized and Generic LLM Pipelines for Dataset Reference Extraction in Scientific Publications

Mengqi Liu
New York University
New York, USA
ml9543@nyu.edu

Jintong Li
New York University
New York, USA
jl13640@nyu.edu

Bo Yu
New York University
New York, USA
by2566@nyu.edu

ABSTRACT

Scientific publications routinely deposit experimental data into public repositories, yet the dataset accession identifiers and URLs that point at those deposits are scattered throughout each paper. Automated extraction of these references is a prerequisite for any data discovery service over the scientific literature. We compare four configurations of two LLM-driven extraction systems on a curated benchmark of 21 papers (DataRef-EXP, 47 ground-truth records) and a representative benchmark of 1,242 papers (DataRef-REV, 1,883 records). DataGatherer is a specialized tool with two retrieval strategies. DocETL is a general purpose declarative document processing framework. All four configurations call the same Kimi K2 model so that observed differences reflect pipeline architecture, with model capability held fixed. Our headline finding is that the dominant variable for accuracy on the larger benchmark is whether the pipeline reads the entire paper, with the section based retrieval strategy losing 22 F1 points on REV due to a structural assumption about HTML markup. Our DocETL pipeline with an iterated 78 line prompt reaches 82.7 percent loose F1 on REV, which is the highest of the four configurations, while the cheapest section based strategy costs 65 times less per paper but caps at 60.6 percent F1. We provide the full code, prompts, ground truth, and per paper outputs at <https://github.com/MengqiLiu-9543/3001de-project> so that the numbers reported here can be reproduced end to end.

KEYWORDS

dataset reference extraction, large language models, document processing, scientific publications, reproducibility

1 INTRODUCTION

Open data is now a baseline expectation in many scientific fields, and most journals require authors to deposit raw datasets into a public repository and cite the accession identifier in the paper. The downstream consumers of these deposits are dataset search engines, repository curators, funder reporting systems, and meta analysis pipelines. All of them need a reliable way to recover the link between a paper and the data it produced. The link is supposed to live inside the paper, in the form of a small string such as PXD047284 (a ProteomeXchange identifier) or GSE10327 (a Gene Expression Omnibus identifier), or in the form of a direct URL to a record. In practice, those strings appear in many places. Some papers put them in a dedicated data availability statement. Others embed them inside a methods sub section about a specific assay. A small number cite them only as a parenthetical inside a results paragraph or as a row in a key resource table.

Until recently, recovering these references at scale required hand crafted regex patterns and rule based parsers, both brittle to journal style and to non standard accession formats. The rise of capable open weight large language models (LLMs) has made it tempting to replace these brittle pipelines with a single LLM prompt that reads each paper and emits a structured list of references. This raises an empirical question. Should a practitioner build on top of a specialized extraction tool that wraps an LLM with domain specific scaffolding, or use a general purpose LLM document processing framework with a custom prompt, or both?

This project compares the two ends of that spectrum on the same input papers, with the same underlying LLM, on two benchmarks of very different sizes. We treat the comparison as an empirical question about pipeline architecture, holding model capability fixed across all configurations. We also treat reproducibility as a first class deliverable. The complete code, prompts, ground truth, and per paper outputs are available in our public GitHub repository, and the README describes a one command path from a fresh clone to a full reproduction of the EXP numbers reported in this paper.

The contributions of this report are as follows. First, we provide a head to head comparison of DocETL [2] and DataGatherer [1] on identical input papers under a single shared LLM backend. Second, we surface and analyze the failure modes of each pipeline on a 1,242 paper benchmark, with a focus on a structural fragility in DataGatherer’s retrieval strategy that causes a sharp recall drop on real world data. Third, we describe a small number of engineering interventions that were necessary to make either pipeline run on a Chinese commercial LLM endpoint, including a vendor patch and a content moderation work around. Fourth, we release everything required to reproduce the numbers, including the v1 prompt, the ground truth files, and an N equals 3 reproducibility study on the smaller benchmark.

2 PROBLEM FORMULATION

Input. A scientific publication available as PubMed Central (PMC) HTML or as JATS XML. We use the same set of papers for all systems and pre fetch the HTML using the NCBI E-utilities API [5].

Output. An ordered list of *dataset reference records*. A record is a tuple of three optional strings, where at least one must be present:

- `dataset_identifier` (a repository specific accession such as PXD047284, GSE10327, phs001049.v1.p1, or a DOI for record level repositories such as Zenodo);
- `repository` (the canonical short name of the hosting repository, for example PRIDE, GEO, dbGaP);

- `url` (a direct URL to the record landing page when one is given in the paper).

Evaluation. We compute precision, recall, and F_1 at the record level. The matching procedure is greedy bipartite. For each paper, every prediction record is converted to a frozen set of (`id_key`, `repo_key`) signatures, and the same is done for every ground truth record. A prediction matches a ground truth if their signature sets intersect under one of two modes:

- **Strict** requires both identifier and repository to match after canonicalization of repository aliases, for example GEO, Gene Expression Omnibus, and NCBI GEO all canonicalize to GEO.
- **Loose** requires only the identifier or the URL to match.

We adopt loose F_1 as the headline metric throughout this report. The strict to loose gap on EXP averages 14.4 percentage points across the four configurations and varies from 9.5 to 21 points by configuration, which means that strict comparisons confound extraction quality with prompt label conventions. Identifiers are routable on their own, since PXD047284 resolves to ProteomeXchange whether the system labelled the repository PRIDE or ProteomeXchange, and this matches the downstream user value of these systems.

Cost. We track LLM cost in USD, using the published Kimi K2 prices of \$0.60 per million prompt tokens and \$2.50 per million completion tokens at the time of this writing.

Engineering effort. We informally track lines of pipeline code and lines of prompt text as a coarse proxy for the human effort required to produce or maintain each system.

3 RELATED WORK

The most direct related work is DataGatherer [1], a specialized open source tool for dataset reference extraction from scientific literature. DataGatherer combines XPath based section retrieval with LLM extraction, and ships with two strategies. The Retrieve Then Read (RTR) strategy uses a curated XPath ruleset to locate a paper’s data availability statement and sends only that snippet to the LLM. The Full Document Read (FDR) strategy sends the entire paper to the LLM in one call. We use both strategies as baselines in this report.

DocETL [2] is a declarative LLM document processing framework that targets complex unstructured document workflows. Pipelines are described in a YAML configuration file, with each operation declaring its model, prompt template, and output schema. DocETL is not specialized for dataset extraction or for scientific text. It serves as our representative of the generic framework approach. The framework was originally designed and benchmarked on legal, medical, and policy document corpora.

Older work in dataset reference extraction relied on regular expressions over known accession patterns and on small classifiers trained on labelled sentences [6, 7]. These approaches achieve high precision on the patterns they cover but degrade quickly on novel repositories and on identifiers that share a prefix with non dataset entities such as gene symbols. The recent shift towards LLM based extraction is partly a response to this brittleness.

The benchmarks we use are released with the DataGatherer paper [1] and are hosted on Zenodo [3]. DataRef-EXP is a small expert curated set of 21 papers with 47 ground truth records, intended for prompt iteration and qualitative analysis. DataRef-REV is a larger random sample of 1,242 PMC papers with 1,883 ground truth records and is intended for system level evaluation at scale.

4 METHODS, ARCHITECTURE, AND DESIGN

4.1 Pipelines under comparison

We compare four configurations across two systems. Table 1 summarizes the configurations.

Configuration	System	Strategy
DG-RTR	DataGatherer	XPath data availability section + LLM
DG-FDR	DataGatherer	Full document + LLM
DocETL v0	DocETL	27 line prompt, full document
DocETL v1	DocETL	78 line iterated prompt with repository catalog

Table 1: Configurations evaluated in this report.

4.2 Shared LLM backend

All four configurations call the same LLM, namely Kimi K2 `kimi-k2-0905-preview` from Moonshot AI. The model is non reasoning, supports a 128k token context window, and is OpenAI API compatible. Holding the LLM fixed isolates the comparison from model capability and forces all observed differences onto pipeline architecture, prompt design, and surrounding scaffolding.

DataGatherer ships with native support for OpenAI, Gemini, and Portkey but not for Moonshot Kimi. We added a small compatibility patch (`patches/datagatherer-kimi-support.patch`) that wires the existing OpenAI client through to the Moonshot endpoint. The patch does not modify any prompt or any retrieval logic.

4.3 DataGatherer details

DataGatherer is run through a wrapper script (`scripts/run_datagatherer.py`) that iterates over the ground truth paper list, calls the library’s `process_articles` method per paper, and serializes one JSON line per paper. The wrapper also overrides two library defaults that conflict with our setup. First, the default `process_articles` flag silently disables full document mode, so we pass it explicitly. Second, the default few shot prompt is RTR style and instructs the model to return one dataset per response, which truncates output for FDR. We switch to the `GPT_FDR_FewShot` prompt template under FDR.

The XPath ruleset that the RTR strategy uses to locate the data availability section lives at `data-gatherer/data-gatherer/config/retrieval_patterns.json`. The ruleset matches `<section>` elements whose `h2` or `h3` text contains a small set of phrases such as `Data Availability`, `Data Sharing`, or `Code Availability`. We did not modify this ruleset.

4.4 DocETL details

DocETL exposes a small library of operations that can be composed into a pipeline. The most common are `map`, which applies an

LLM prompt to each input document, reduce and resolve, which aggregate or deduplicate records, and split paired with gather, which break a long document into chunks and recombine per chunk results. The framework also ships an agentic optimizer that can rewrite a pipeline automatically. Our pipeline is a single map operation that takes one paper at a time and emits a list of dataset references. We chose this minimal configuration deliberately. The longest paper in DataRef-EXP is roughly 50k tokens and the longest in DataRef-REV is roughly 69k tokens, so every paper fits comfortably inside Kimi K2’s 128k context window and split or gather chunking is unnecessary. A single map call also keeps the DocETL configurations in the same single shot full document orchestration as DG-FDR.

DocETL is run in process through a wrapper script (`scripts/run_docetl.py`) that loads the YAML pipeline, runs DocETL’s DSLRunner, and converts the output into the same JSON line format as DataGatherer. The wrapper installs two LiteLLM hooks. The first is a success callback that accumulates per call token usage, which we use to compute USD cost from token counts. The second is a wrapper around `litellm.completion` that catches Moonshot `content_filter` rejections and returns a synthetic empty references response. Without this wrapper, a single rejected paper would raise an exception and abort the entire batch.

We compare two prompts. Pipeline v0 uses a 27 line prompt that simply asks the model to extract dataset references and return them in a fixed schema. Pipeline v1 uses a 78 line prompt that contains a 27 entry repository identifier catalog with regex patterns and a small number of explicit field formatting rules. The catalog covers the NCBI family (GEO, SRA, BioProject, dbGaP, PDB), the proteomics family (PRIDE, ProteomeXchange, MassIVE, iProX, jPOST, PDC, PanoramaPublic), the cancer genomics family (CPTAC, TCGA, DepMap, ICGC), the European family (EGA, ArrayExpress, ENA, DDBJ, BioStudies), and the generic DOI backed repositories (Zenodo, Figshare, Mendeley, Dryad, OSF). The prompt was iterated on the EXP development set only, before any REV results were observed.

4.5 Reproducibility design

We treat reproducibility as a primary deliverable. The repository contains the YAML pipelines, the ground truth CSV files for both benchmarks, the per paper output JSON for every reported configuration, the metric computation scripts, and an N equals 3 reproducibility study on EXP. The README describes a five step path from a fresh clone to a full EXP reproduction in roughly five minutes of wall time and \$1.25 of LLM spend. The only required user input is a Kimi API key, which the user obtains from <https://platform.moonshot.cn> and pastes into a local `.env` file. The key is never committed.

5 RESULTS

5.1 DataRef-EXP, N equals 3 runs per system

Table 2 reports loose precision, loose recall, and loose F_1 as mean and standard deviation across three independent runs. We cleared the LiteLLM disk cache between runs to ensure independent calls.

System	Preds	P	R	F1
DG-RTR	36.0 ± 1.7	95.4 ± 1.4	73.1 ± 2.5	82.7 ± 1.1
DG-FDR	37.7 ± 2.9	93.3 ± 6.8	74.5 ± 0.0	82.7 ± 2.8
DocETL v0	37.3 ± 3.1	85.1 ± 5.3	67.4 ± 2.5	75.1 ± 1.9
DocETL v1	47.0 ± 2.0	80.8 ± 0.8	80.9 ± 4.3	80.8 ± 2.5

Table 2: DataRef-EXP results, N equals 3, mean ± standard deviation. All four numbers are percentages except the prediction count.

On EXP the three full document configurations (DG-RTR, DG-FDR, DocETL v1) are statistically indistinguishable at F_1 around 81 percent, with confidence intervals that overlap. DG-RTR wins on cost, at \$0.011 for 21 papers, which is roughly 27 times cheaper than DocETL v1 and 54 times cheaper than DG-FDR. Prompt iteration on DocETL adds 5.7 F_1 points relative to v0 on EXP, almost entirely on the recall side (67 to 81 percent). The standard deviation of around 2 to 3 F_1 points across runs is the LLM stochasticity floor at temperature=0, since we observed token level differences in completions across runs at fixed temperature.

5.2 DataRef-REV, single run

Table 3 reports loose metrics for a single run on the larger benchmark. DG-FDR was budget capped at \$37 of LLM spend.

System	Preds	P	R	F1
DG-RTR	1,275	75.06	50.82	60.61
DG-FDR	2,149	76.17	86.94	81.20
DocETL v1	2,612	71.13	98.67	82.67

Table 3: DataRef-REV results, N equals 1 over 1,242 papers. All numbers are percentages except the prediction count.

DG-RTR collapses on REV. Its F_1 drops by 22 points relative to EXP, while DG-FDR and DocETL v1 hold their EXP performance within a couple of points. The collapse is driven by a structural assumption in the XPath ruleset. The ruleset matches `<section>` elements whose h2 or h3 contains a phrase such as Data Availability. In our REV sample, 51.6 percent of papers (641 out of 1,242) do not have any such section. Most of those papers embed accessions inside Methods sub sections such as MS Data Analysis or Proteomics Data Acquisition without a dedicated data availability heading. Those 641 papers contain 899 ground truth records, which is 47.7 percent of all REV ground truth, and DG-RTR can never see those records. This is a hard recall ceiling that prompt iteration cannot lift.

DG-FDR and DocETL v1 do not have this limitation because they read every section. Among the two, DG-FDR has slightly higher precision (76.17 vs 71.13) and DocETL v1 has higher recall (98.67 vs 86.94) and slightly higher F_1 . The precision gap is consistent with v1’s prompt design, which contains an explicit catalog of repositories and rules and tends to extract reused public datasets cited from prior work. We analyze this in the next subsection.

5.3 Cost vs accuracy trade off on REV

Figure 4 shows the cost trade off on REV. The economic conclusions differ depending on use case.

System	Cost (USD)	F1
DG-RTR	0.50	60.61
DocETL v1	17.67	82.67
DG-FDR	32.47	81.20

Table 4: Cost and F_1 on REV (1,242 papers).

For an industrial scale ingestion pipeline that processes hundreds of thousands of papers per month, only DG-RTR is economically viable, but the 51 percent recall is too low for many use cases. For a research pipeline operating on a curated corpus of a few thousand papers, DocETL v1 is the cost effective choice at \$17.67 for 1,242 papers and 82.67 percent F_1 . DG-FDR pays roughly twice the LLM cost for 1.5 fewer F_1 points and is dominated by DocETL v1 in this regime.

5.4 Failure mode analysis

We manually inspected every false positive that DocETL v1 produced on EXP, which is 9 records. The categorization is shown in Table 5.

Failure category	Count
Reused public datasets cited from prior work	5
Ground truth missing the paper entirely	1
Field mapping error (URL placed in identifier)	2
Synonymous mirror (jPOST mirrored as PXD)	1
True hallucination	0

Table 5: DocETL v1 false positive categorization on EXP.

The first category, reused public datasets, accounts for 5 of 9 false positives. These are accessions cited in a paper’s Methods section as part of an upstream dataset that the authors did not deposit themselves. PMC11066909 is a typical example, citing four GEO accessions and one Proteomic Data Commons accession from prior work, while the ground truth credits only the single ProteomeXchange identifier that the authors themselves deposited. The v1 prompt extracts these because they are concrete accessions that match the regex catalog. We inspected the corresponding DG-FDR runs and found that DG-FDR run 2 picks up the same five accessions, which suggests this is a benchmark scoping decision and not an extraction error. The other two DG-FDR runs do not pick them up, which is consistent with the precision standard deviation of 6.8 points we observed for DG-FDR on EXP.

The second category is a single paper, PMC11015306, where the body text clearly states a ProteomeXchange deposit but the ground truth has no record for that paper at all. DG-FDR also extracts the same identifier across all three runs, which we treat as confirming evidence that the ground truth is incomplete here.

The third category is two cases of mapping a portal URL into the identifier field, for example PMC8628860 which references the

HTAN data portal and an MSKCC URL but does not give an accession. The fourth category is a single case of a synonymous mirror, where jPOST identifier JPST002506 mirrors ProteomeXchange identifier PXD049309. The paper cites both and DG-RTR also extracts JPST002506.

We did not find any case of an identifier that does not appear in the paper text, which is the textbook definition of LLM hallucination on this task. The headline precision number on EXP underestimates v1’s actual extraction quality by treating ground truth scoping decisions as model errors.

5.5 Engineering observations

We surface two engineering issues that are not visible from the metric tables.

The first is Moonshot’s commercial content moderation. When a generic full document pipeline sends 25k tokens of biomedical text, it occasionally trips Moonshot’s content classifier on terms such as HIV, gene editing, low Earth orbit, or paediatric trials, and the API returns a `BadRequestError(content_filter)` on the entire prompt. This affected 2 out of 1,241 REV papers in our DocETL v1 run. We confirmed the issue is upstream of the pipeline by issuing a direct requests call to Moonshot’s API with the offending paper, which returned the same rejection. We worked around it by wrapping LiteLLM’s completion call to convert content filter rejections into empty extraction responses. DG-FDR’s library exception handler does the equivalent. Pipelines that pre scope to small text snippets such as DG-RTR are structurally immune since the smaller payloads do not trigger the classifier.

The second is input format asymmetry. DocETL receives BeautifulSoup stripped PMC HTML, while DataGatherer fetches its own JATS XML through NCBI E-utilities. Both formats ultimately encode the same paper content, but the noise profiles are different. A more careful study would normalize both pipelines to identical input bytes.

5.6 Limitations

The study has eight limitations that we make explicit. (L0) DataRef is one benchmark family. Generalization to bioRxiv, ICML, or non biomedical corpora is future work. (L1) All four configurations use the same LLM. A different model such as GPT-4o might shift relative rankings. (L2) Ground truth incompleteness, as documented in the failure mode analysis, would raise both systems’ precision under manual audit. (L3) Prompt engineering effort is asymmetric. DocETL v1 has a hand tuned 78 line prompt. DataGatherer is run with stock few shot templates. Equivalent prompt iteration on DG would likely close the v1 recall gap. (L4) LLM stochasticity at temperature=0 produces around 3 F_1 points of run to run variation. (L5) Commercial content moderation affects 2 of 1,241 REV papers in our run. (L6) Input format asymmetry as described in the engineering observations subsection. (L7) REV is N equals 1 due to the cost of running the larger benchmark, so its absolute numbers carry similar uncertainty as EXP.

5.7 Summary of findings

The framing of specialized versus generic turned out to be the wrong axis. The dominant variable for accuracy on the larger benchmark is whether the pipeline reads the entire paper. The retrieval based strategy that pre scopes to a data availability section is 65 times cheaper but loses 22 F_1 points on REV due to a structural assumption about HTML markup that holds only in 48 percent of papers. The full document strategies, whether implemented in the specialized tool or in the generic framework with an iterated prompt, perform within 1.5 F_1 points of each other and within 2 F_1 points of the EXP numbers. The generic framework with an iterated prompt is the cost effective choice in our experiments at \$17.67 for 1,242 papers and 82.67 percent loose F_1 . The complete code, prompts, ground truth, and per paper outputs that produced these numbers are available at <https://github.com/MengqiLiu-9543/3001de-project>.

REFERENCES

- [1] M. Marini, A. Brown, S. Castro Pena, S. Castro Reyes, J. Freire. Data Gatherer. *GitHub repository*, 2025. <https://github.com/VIDA-NYU/data-gatherer>.
- [2] S. Shankar, A. G. Parameswaran, E. Wu. DocETL: Agentic Query Rewriting and Evaluation for Complex Document Processing. In *Proceedings of the VLDB Endowment*, 2025. <https://docetl.org>.
- [3] DataRef-EXP and DataRef-REV benchmark dataset, 2024. Zenodo. <https://doi.org/10.5281/zenodo.15549086>.
- [4] Moonshot AI. Kimi K2 model card. 2025. <https://platform.moonshot.cn>.
- [5] E. W. Sayers et al. Database resources of the National Center for Biotechnology Information. *Nucleic Acids Research*, 38(suppl_1), 2010.
- [6] A. Singh, K. Kelley, S. Mukherjee. Dataset name extraction from scientific text. *arXiv preprint arXiv:1808.05272*, 2018.
- [7] A. Prasad, M. Y. Kan. Dataset mention extraction and classification. In *Proc. of the Workshop on Extracting Structured Knowledge from Scientific Publications*, 2019.
- [8] M. D. Wilkinson et al. The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data*, 3, 2016.
- [9] LiteLLM. Unified interface for 100+ LLMs. 2024. <https://github.com/BerriAI/lite>.